OjoAlBondi

Documentación de base de datos

Supabase y SQL: estructura, instalación, scripts y problemas resueltos

Dato	Valor
Proyecto	OjoAlBondi — gestor de reclamos del transporte público de Pilar
Materia	Aplicaciones Móviles 2026 — Universidad Nacional de Pilar
Fecha	06/10/2026
Versión de la base	Instalación completa (equivalente a las migraciones 001 a 005)
Especificación	Versión 10 + Registros de requerimiento N° 29 a 31
Plataforma	Supabase: PostgreSQL, Auth y Storage

Contenido

1. Resumen 3

2. Claves y configuración 3

3. Estructura de la base 4

3.1 Tablas 4

3.2 Vistas 4

3.3 Funciones y reglas 5

3.4 Flujo de estados 5

3.5 Permisos 5

3.6 Fotos 6

4. Instalación en un proyecto de Supabase 6

Paso 1 — Reiniciar (solo si la base ya tenía tablas) 6

Paso 2 — Instalación completa 6

Paso 3 — Paradas 6

Paso 4 — Datos de prueba 6

Base que ya tenía las migraciones 001 a 003 6

5. Datos de prueba y administración 7

6. Problemas encontrados y soluciones 7

7. Consultas útiles 8

8. Registros de cambio de la base de datos 8

1. Resumen

La aplicación usa **Supabase** como backend: una base PostgreSQL con seguridad por filas (RLS), autenticación (Auth) y almacenamiento de fotos (Storage). Las reglas de negocio y los permisos se validan **en la base**, así no se pueden saltear desde fuera de la app.

Todo lo de la base está en `ojoalbondi-backend/`:

Archivo o carpeta	Para qué sirve
supabase/instalacion_completa.sql	Crea la base completa en un proyecto nuevo. Es el único archivo que hace falta para instalar.
supabase/esquema/01 a 05	La misma base separada por tema (tablas, funciones, vistas, seguridad y datos iniciales). instalacion_completa.sql se genera a partir de estos archivos.
supabase/migraciones/001 a 005	Historial de cambios para bases ya creadas. En una base nueva no se usan.
supabase/utilidades/reiniciar_base.sql	Borra todo lo del proyecto para reinstalar. Destructivo.
supabase/paradas_seed.sql	Paradas de colectivo. Lo genera scripts/importar-paradas-osm.mjs.
scripts/armar-instalacion.mjs	Une los archivos de esquema en instalacion_completa.sql (npm run armar-instalacion).
scripts/importar-paradas-osm.mjs	Descarga las paradas de OpenStreetMap (npm run paradas).
scripts/datos-prueba.mjs	Cuentas, suscripciones y reclamos de prueba (npm run datos-prueba / datos-prueba:borrar).

2. Claves y configuración

Clave	Empieza con	Dónde va	¿Secreta?
Publicable	sb_publishable_	ojoalbondi-frontend/.env (EXPO_PUBLIC_SUPABASE_ANON_KEY)	No: va dentro de la app; los permisos de la base limitan qué puede hacer cada usuario.
Secreta (o service_role legacy)	sb_secret_ / eyJ...	Solo en la terminal, para datos-prueba.mjs ($env:SUPABASE_SERVICE_ROLE_KEY)	Sí: saltea todos los permisos. Nunca en archivos, GitHub ni capturas. Si se expone, se elimina y se crea otra.

Archivo `ojoalbondi-frontend/.env` (no se sube a GitHub):

```
EXPO_PUBLIC_SUPABASE_URL=https://bmtzespsvdafhlrwnpjp.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=sb_publishable_...
```

Durante el desarrollo, **Authentication → Sign In / Providers → Email → "Confirm email"** está desactivado. Se vuelve a activar antes de usar la app con personas reales.

3. Estructura de la base

3.1 Tablas

Tabla	Contenido
organizaciones	Empresas de transporte (6) y municipio.
organizaciones_contacto	Razón social, dirección y teléfono. Solo uso administrativo: la app no la puede leer.
perfiles	Datos de cada cuenta: nombre, apellido, fecha de nacimiento, rol (ciudadano, empresa, municipio, admin) y organización.
lineas	Las 7 líneas, con la empresa que la opera (dato informativo).
lugares	Catálogo de 33 lugares con nombre general (25 destinos y 8 de paso).
ramales	Los 30 ramales: línea, código, lugar A y lugar B.
ramal_paso	Lugares intermedios de cada ramal, en orden.
motivos	Los 8 motivos de reclamo, con su descripción y si es grave.
paradas	Paradas de colectivo de OpenStreetMap: nombre, líneas, latitud y longitud.
suscripciones	Qué empresa gestiona cada línea (una sola activa por línea).
reclamos	Ticket, ciudadano, línea, sentido, motivo, interno o patente, descripción, fecha y hora del hecho, ubicación, parada, foto, estado y fecha de registro.
historial_estados	Cada cambio de estado: estado anterior y nuevo, comentario, responsable y fecha. Inmutable.
tickets_contador	Contador para el ticket correlativo por año (OB-2026-0001…).

3.2 Vistas

- **sentidos_por_linea:** los destinos de los ramales de cada línea, sin repetir. De ahí sale el campo Sentido.
- **reclamos_gestion:** los reclamos que gestiona la institución conectada, con la descripción del motivo, el sentido y solo nombre y apellido del ciudadano.

3.3 Funciones y reglas

Función	Qué hace
crear_perfil_al_registrarse	Crea el perfil al registrarse. Siempre como ciudadano (RN-02).
proteger_rol	Impide cambiarse el rol o la organización desde la app.
mi_rol, mi_organizacion, mis_lineas, puede_gestionar	Permisos: el municipio gestiona todas las líneas; la empresa, las de sus suscripciones activas.
validar_suscripcion	Solo las empresas se suscriben; registra quién aprobó y cuándo.
generar_ticket, asignar_ticket	Ticket correlativo por año, sin repeticiones.
validar_sentido	El sentido elegido tiene que ser un destino de la línea del reclamo.
registrar_estado_inicial	Agrega la entrada "Recibido" al historial de cada reclamo nuevo.
historial_inmutable	Rechaza modificar el historial, incluso como administrador.
cambiar_estado	Única forma de cambiar un estado: valida permiso, flujo y motivo obligatorio al anular.
contar_bandeja	Cantidad de reclamos por estado, en una sola consulta.
calor_por_parada, reclamos_de_parada	Mapa de calor: totales por parada y últimos 20 reclamos (motivo, fecha, línea). Sin anulados, máximo 3 por ciudadano y parada, filtros por motivo, período y línea.
es_admin, admin_listar_cuentas, admin_asignar_rol	Administración: listar cuentas y asignar roles. Solo el administrador.

3.4 Flujo de estados

Desde	Hacia	Quién	Condición
(nuevo)	Recibido	Sistema	Al crearse el reclamo
Recibido	En revisión	Institución	Al tomarlo
En revisión	Atendido	Institución	Comentario recomendado
Recibido o En revisión	Anulado	Institución	Motivo obligatorio

Atendido y Anulado son estados finales.

3.5 Permisos

Perfil	Puede
Ciudadano	Crear y ver sus reclamos; ver el mapa de calor de todas las líneas (solo totales).
Empresa	Ver y gestionar los reclamos de las líneas con suscripción activa; mapa de calor de esas líneas.
Municipio	Ver y gestionar los reclamos de todas las líneas.
Administración	Gestionar suscripciones y roles de las cuentas; ver el mapa.
Nadie desde la app	Editar reclamos directamente, modificar el historial, cambiarse el rol ni leer los contactos de las empresas.

3.6 Fotos

Bucket privado **fotos-reclamos**. Cada ciudadano sube a su carpeta y ve solo sus fotos; la institución ve las de los reclamos de sus líneas.

4. Instalación en un proyecto de Supabase

Cada archivo se corre en **SQL Editor → New query**: pegar el contenido completo y tocar **Run**. Todos se pueden repetir sin romper nada, pero el orden importa. **Desactivar el traductor de Chrome** en supabase.com: rompe el editor.

Paso 1 — Reiniciar (solo si la base ya tenía tablas)

`supabase/utilidades/reiniciar_base.sql`. Verificación (no tiene que devolver filas):

```
select table_name from information_schema.tables where table_schema = 'public';
```

Paso 2 — Instalación completa

`supabase/instalacion_completa.sql`. Verificación:

```
select (select count(*) from lineas), (select count(*) from ramales),
       (select count(*) from motivos), (select count(*) from organizaciones);
-- Resultado esperado: 7, 30, 8, 7
```

Paso 3 — Paradas

`supabase/paradas_seed.sql` (si no existe, generarlo con `npm run paradas`). Si el editor se traba, correr los bloques de a uno.

```
select count(*) from paradas;   -- mayor que 0
```

Paso 4 — Datos de prueba

Desde `ojoalbondi-backend`, en PowerShell:

```
npm install
$env:SUPABASE_URL="https://bmtzespsvdafhlrwnpjp.supabase.co"
$env:SUPABASE_SERVICE_ROLE_KEY="sb_secret_..."
npm run datos-prueba:borrar
npm run datos-prueba
```

Después, cerrar la terminal para que la clave no quede cargada.

Base que ya tenía las migraciones 001 a 003

Alternativa a la reinstalación: `npm run datos-prueba:borrar` → migración 004 → migración 005 → `npm run datos-prueba`.

5. Datos de prueba y administración

Cuenta (contraseña Prueba123!)	Perfil	Ve
vecino1 ... vecino8@prueba.ojoalbondi	Ciudadano	Sus reclamos
tratado@prueba.ojoalbondi	Empresa	Línea 501 (suscripción de prueba)
pilarbus@prueba.ojoalbondi	Empresa	Línea 510 (suscripción de prueba)
municipio@prueba.ojoalbondi	Municipio	Todas las líneas
admin@prueba.ojoalbondi	Administración	Suscripciones y cuentas

La línea 503 queda sin suscripción a propósito: sus reclamos los ve solo el municipio (RF-25). El primer administrador real se crea desde el SQL Editor; después asigna los roles desde la app:

```
update perfiles set rol = 'admin', organizacion_id = null where id = '<id del usuario>';
```

Suscripciones desde SQL (también se gestionan desde la app, perfil Administración):

```
insert into suscripciones (linea, organizacion_id, estado) values ('503', 'tratado', 'activa');
update suscripciones set estado = 'suspendida' where linea = '503' and estado = 'activa';
```

6. Problemas encontrados y soluciones

Problema	Causa	Solución
"Overpass respondió 406" en el script de paradas	OpenStreetMap rechaza pedidos sin identificación	El script envía un User-Agent propio y prueba tres servidores (Cambio N° 12)
PowerShell: "El término https://... no se reconoce"	Faltaban comillas	$env:VARIABLE="valor"
"This endpoint requires a valid Bearer token"	Se usó la clave publicable en lugar de la secreta	Usar sb_secret_ o service_role
"Invalid API key"	Clave secreta copiada incompleta o expuesta	Eliminarla, crear una nueva y copiarla con el botón de copiar
"Missing script: datos_prueba:borrar"	Guion bajo en lugar de guion medio	npm run datos-prueba:borrar
"Ya hay datos de prueba cargados"	reiniciar_base.sql no borra las cuentas	Correr antes npm run datos-prueba:borrar
El panel de Supabase se rompe ("removeChild")	Traductor de Chrome	"No traducir nunca este sitio" o ventana de incógnito
"column reclamo_id does not exist" al instalar	Quedaban tablas viejas: reiniciar_base.sql no se había ejecutado	Correr reiniciar_base.sql, verificar que no queden tablas y reinstalar
La migración 003/004 se detiene	Hay cuentas o reclamos con el formato anterior	Correr antes npm run datos-prueba:borrar

7. Consultas útiles

```
-- Reclamos por estado y por línea
select estado, count(*) from reclamos group by estado;
select linea, count(*) from reclamos group by linea order by linea;

-- Historial de un reclamo
select * from historial_estados where reclamo_id = '<id>' order by creado_en;

-- Suscripciones activas
select linea, organizacion_id, aprobada_en from suscripciones where estado = 'activa';

-- Cuentas que no son ciudadanos
select id, nombre, apellido, rol, organizacion_id from perfiles where rol <> 'ciudadano';

-- Mapa de calor (30 días, todas las líneas)
select * from calor_por_parada(null, 30, null) order by cantidad desc limit 10;
```

8. Registros de cambio de la base de datos

N°	Fecha	Qué se hizo
1	28/09/2026	Esquema inicial: perfiles, reclamos, bucket de fotos y permisos del ciudadano.
4	28/09/2026	Tabla paradas y script de importación desde OpenStreetMap.
8	28/09/2026	Migración 001: estado "Recibido" e interno opcional solo para robo.
9	28/09/2026	Migración 002: Etapa 1 (organizaciones, roles, historial, cambio de estado, permisos).
11	29/09/2026	Script reiniciar_base.sql.
12	01/10/2026	Script de paradas: corrección del error 406.
14	02/10/2026	Migración 003: datos reales, catálogo de lugares, ramales, suscripciones, ticket correlativo, optimización.
15	05/10/2026	Migración 004 e instalación completa: nombres consistentes (reportes → reclamos), tabla motivos, esquema separado por tema.
16	05/10/2026	Migración 005: mapa de calor y administración (suscripciones y cuentas).